



Cairo University
Faculty of Computers and Artificial Intelligence

CS213
Object Oriented Programming

Vole Machine Simulator

Object-Oriented Implementation Report

Under the supervision of
Prof. Mohammad El-Ramly

Team Members

Name	ID	Phone Number (Team Leader)
Muhammad Ahmed Noureldin Abdelghany *Team Leader*	20251324	01114102049
Nabil Mohamed Bahget Mohamed	20250699	—
Abdelrahman Haithem Mostafa Said	20250378	—
Eslam Ahmed Abdelaziz Ahmed	20250083	—

November 2026

1. Introduction

This report documents the implementation of the Vole Machine Simulator, the course project of CS213 Object Oriented Programming. The simulator is a C++ program with a Qt Widgets graphical interface. It loads a program written in Vole machine language, executes it one instruction at a time or all at once, and shows the state of the CPU, the registers, the main memory and the screen. Every hardware part of the Vole machine is its own class, and four team members each implemented a part of the machine.

The report is organised as follows. Section 2 explains the Vole machine: its architecture, specification, instruction set, machine cycle and floating-point format. Section 3 presents the software architecture and the class diagram of the implementation. Section 4 describes every class and its members. Section 5 describes the graphical user interface. Section 6 gives the program file format and Section 7 the instructions for building and running the simulator on Windows. Section 8 lists the test programs that were run and their results. Section 9 gives the work breakdown of the team and Section 10 concludes the report.

2. The Vole Machine

2.1 Overview

The Vole machine is a small, hypothetical computer used to teach how a real computer fetches and executes a program. It has a central processing unit (CPU) with a small set of registers, a main memory of 256 one-byte cells, and a simple instruction set of twelve instructions. A program is a list of machine instructions stored in memory; the CPU runs them one after another until it reaches a HALT instruction.

The simulator reproduces this behaviour in software: it loads a program from a text file into memory, runs it step by step or continuously, shows the state of memory, registers and screen, and executes every instruction of the instruction set.

2.2 Specification

Component	Specification
CPU	Contains the 16 general-purpose registers, the program counter, the instruction register, the ALU and the control unit.
General-purpose registers	16 registers numbered 0 to F (hexadecimal), each 8 bits wide.
Instruction register (IR)	Holds the one instruction (16 bits) that is currently being executed.
Program counter (PC)	Holds the memory address of the next instruction to fetch. It starts at 00 and increases by 2 after every fetch.
Main memory	256 cells of 1 byte (8 bits) each, with addresses 00 to FF.
Instruction size	16 bits, stored in two consecutive memory cells. The first hexadecimal digit is the op-code and the other three digits are the operands. Instructions start at even addresses.
Instruction set	Twelve instructions with op-codes 1 to C (see Section 2.3).
Data representation	Integers use two's complement. Real numbers use an 8-bit floating-point format (1 sign bit, 3-bit exponent, 4-bit mantissa).
Screen	Memory-mapped: storing a value into the cell at address FF writes that value to the screen.

2.3 Instruction set

Every instruction is four hexadecimal digits. In the formats below, R, S and T are register numbers, and XY is an 8-bit value or memory address made of the two digits X and Y.

Op	Format	Meaning	Example
1	RXY	LOAD register R with the contents of the memory cell at address XY.	14A3: R4 ← memory[A3]
2	RXY	LOAD register R with the value XY.	20A3: R0 ← A3
3	RXY	STORE the contents of register R in the memory cell at address XY. If XY is FF, the value is written to the screen.	35B1: memory[B1] ← R5
4	0RS	MOVE the contents of register R to register S.	40A4: R4 ← RA
5	RST	ADD registers S and T as two's complement numbers and put the result in R.	5726: R7 ← R2 + R6
6	RST	ADD registers S and T as floating-point numbers and put the result in R.	634E: R3 ← R4 + RE
7	RST	OR registers S and T and put the result in R.	7CB4: RC ← RB OR R4
8	RST	AND registers S and T and put the result in R.	8045: R0 ← R4 AND R5
9	RST	XOR registers S and T and put the result in R.	95F3: R5 ← RF XOR R3
A	R0X	ROTATE register R to the right X times. The bit leaving the low end re-enters at the high end.	A403: rotate R4 right 3 times
B	RXY	JUMP to address XY if register R equals register 0; otherwise continue in sequence. The jump copies XY into the PC.	B43C: if R4 = R0 then PC ← 3C
C	000	HALT execution.	C000: stop

2.4 The machine cycle

The CPU repeats a three-phase cycle until a HALT instruction is executed:

- **Fetch.** The control unit reads the two bytes stored at the address in the PC and places them in the instruction register, then adds 2 to the PC so that it points to the next instruction.
- **Decode.** The control unit splits the instruction register into the op-code (first digit) and the operand fields R, S, T, X and XY that the op-code needs.
- **Execute.** The control unit carries out the instruction: it moves data between memory and registers, asks the ALU to compute a result, writes to the screen, changes the PC (JUMP), or stops the machine (HALT). An op-code that is not recognised also stops the machine.

2.5 Floating-point format

Floating-point values use 8 bits: bit 7 is the sign (1 means negative), bits 6 to 4 are the exponent stored in excess-4, and bits 3 to 0 are the mantissa.

Bit 7	Bits 6-4	Bits 3-0
Sign (1 = negative)	Exponent (excess-4)	Mantissa (4 bits)

- **Decoding:** value = (mantissa / 16) × $2^{(\text{exponent} - 4)}$, negated if the sign bit is set. For example, 7A has sign 0, exponent 7 and mantissa A, so it decodes to (10 / 16) × 2^{3} = 5.0.

- **Encoding:** the value is normalised so that its fraction lies in [0.5, 1), 4 is added to the exponent, and the top 4 bits of the fraction become the mantissa (extra bits are truncated).
- **Limits:** an exponent that overflows is clamped to the largest representable value; an exponent that underflows gives 00.

2.6 Worked example

The following five-instruction program is loaded at address 00. It adds 5 and 3 and shows the result on the screen.

Address	Instruction	Meaning	Effect
00	2105	LOAD R1 with the value 05	R1 = 05
02	2203	LOAD R2 with the value 03	R2 = 03
04	5312	ADD R1 and R2 (two's complement), result into R3	R3 = 08
06	33FF	STORE R3 at address FF	8 is written to the screen
08	C000	HALT	The machine stops

The example uses every part of the machine: memory holds the program, the registers hold the data, the ALU performs the addition, the PC steps through addresses 00, 02, 04, 06 and 08, and the screen is reached through the memory-mapped address FF.

3. Software Architecture

3.1 Overview

The simulator follows an object-oriented design in which each part named in the Vole machine specification is one class. The classes are split over four pairs of header and source files, one pair for each team member, plus one file for the graphical interface. The design is deliberately simple: no class is added that the specification does not need, and every class has one clear purpose.

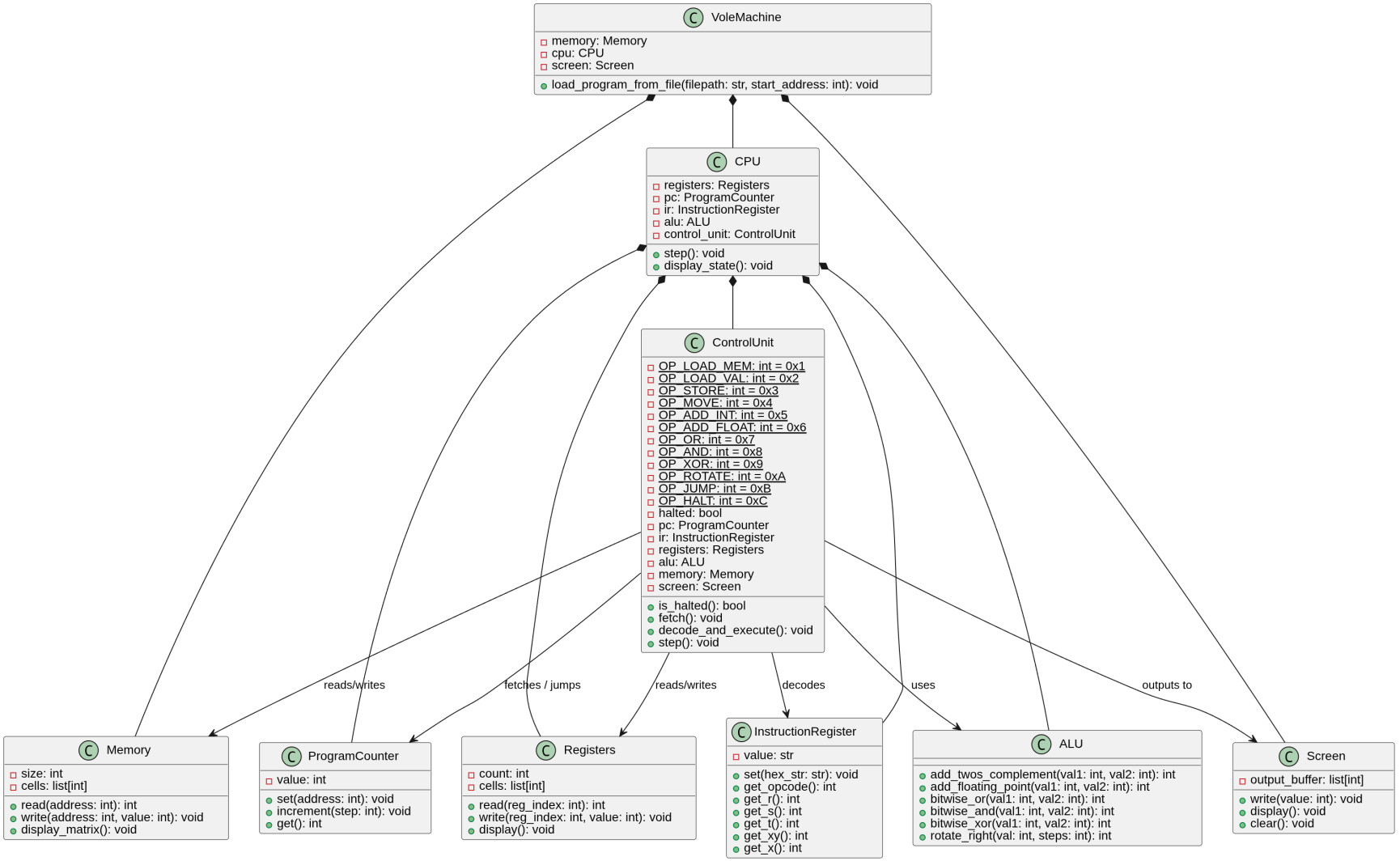
3.2 Source files

File	Contents	Implemented by
Muhammad.h / Muhammad.cpp	ALU, CU (control unit)	Muhammad
Eslam.h / Eslam.cpp	Memory, Screen	Eslam
Abdelrahman.h / Abdelrahman.cpp	programCounter, registers, voleMachine	Abdelrahman
Nabil.h / Nabil.cpp	InstructionRegister, CPU	Nabil
gui.cpp	Qt Widgets graphical interface (MainWindow and the Machine structure)	Muhammad

3.3 Class diagram

Figure 1 shows the ten classes of the simulator with their real attributes and methods. An arrow means that the class holds a pointer to the target class and uses it. The CPU keeps pointers to the registers, the program counter, the instruction register, the ALU and the control unit; the control unit uses the same components, together with the memory and the screen, to run the fetch-decode-execute cycle.

Figure 1. Class diagram of the implemented simulator.



3.4 How the components work together

The program is started from the interface. The following sequence describes what happens when a program is loaded and run:

- **Loading.** voleMachine reads the program file and writes each instruction, two bytes at a time, into Memory starting at address 00. The program counter starts at 00.
- **Stepping.** The interface calls CPU::step(), which calls CU::step(). If the machine is not halted, the control unit runs one cycle.
- **Fetch.** CU::fetch() reads the two bytes at the program counter from Memory, adds 2 to the program counter, builds a four-digit hexadecimal string and stores it in the InstructionRegister.
- **Decode.** CU::decode_and_execute() asks the InstructionRegister for the op-code and for the operand fields (R, S, T, XY and X).
- **Execute.** Depending on the op-code, the control unit reads or writes registers and memory, calls the ALU for arithmetic and logic, writes to the Screen when a value is stored at address FF, changes the program counter for a jump, or sets its halted flag.
- **Display.** After each step, the interface reads the program counter, the registers, the memory and the screen and redraws its panels.

4. Class Descriptions

This section describes each class: its role, its members, and how it works. The table in each subsection lists the members of the class.

4.1 ALU (Muhammad)

The ALU performs all arithmetic and logic on 8-bit values. Every operation masks its result to 8 bits, so values always stay between 00 and FF.

Member	Description
decode(int): float	Converts an 8-bit floating-point value to a C++ float using the sign, excess-4 exponent and 4-bit mantissa.
encode(float): int	Converts a float to the 8-bit format: it records the sign, normalises the value into [0.5, 1) while counting the exponent, adds 4 to the exponent, and keeps the top 4 mantissa bits. Zero encodes to 00, overflow is clamped to the largest value and underflow gives 00.
add_twos_complement(a, b)	Integer addition masked to 8 bits.
add_floating_point(a, b)	Decodes both operands, adds them as floats and encodes the result.
bitwise_or / and / xor(a, b)	Bitwise operations, masked to 8 bits.
rotate_right(val, steps)	8-bit circular right rotation. The step count is taken modulo 8.

4.2 CU - Control Unit (Muhammad)

The control unit drives the fetch-decode-execute cycle. It holds pointers to the program counter, the instruction register, the memory, the registers, the ALU and the screen, and a halted flag. The twelve op-codes are named constants (OP_LOAD_MEM = 0x1 up to HALT = 0xC) instead of bare numbers.

Member	Description
is_halted()	Reports whether the machine has halted.
fetch()	Reads two bytes from memory at the program counter, advances the program counter by 2, builds a four-digit uppercase hexadecimal string and stores it in the instruction register.
decode_and_execute()	Reads the op-code and operand fields and carries out the instruction: LOAD from memory or with a value; STORE to memory, or to the screen when the address is FF; MOVE; two's complement and floating-point ADD; OR, AND, XOR; ROTATE; and JUMP, which sets the program counter when register R equals register 0. Any other op-code, including HALT, sets the halted flag.
step()	Does nothing if the machine is halted; otherwise runs fetch() followed by decode_and_execute().

4.3 Memory (Eslam)

Memory is a fixed array of 256 integer cells, all initialised to 0.

Member	Description
read(address)	Returns the value of one cell. An address outside 0 to 255 ends the program.
write(address, value)	Stores a value in one cell, with the same address check.
display_matrix()	Prints every cell with its address in hexadecimal.

4.4 Screen (Eslam)

The screen collects everything the program writes to address FF in an output buffer.

Member	Description
write(value)	Appends a value to the output buffer.
display()	Prints the buffered values separated by spaces.
clear()	Empties the buffer.

4.5 programCounter (Abdelrahman)

The program counter holds the address of the next instruction.

Member	Description
set(address)	Sets the address, for example to 00 at start-up or to the target of a jump.
get()	Returns the current address.
increment()	Adds 2 to the address, because every instruction is two bytes long.

4.6 registers (Abdelrahman)

The register file contains the sixteen general-purpose registers, stored in a vector and initialised to 0.

Member	Description
read(index)	Returns the value of one register.
write(index, value)	Stores a value in one register.
display()	Prints a table of all registers in hexadecimal.

4.7 voleMachine (Abdelrahman)

The voleMachine class connects the machine to the outside world. It holds pointers to the memory, the CPU and the screen.

Member	Description
load_program_from_file(path, address)	Reads a text file line by line. Blank lines and lines that start with ";" are skipped. The first four hexadecimal digits of every other line are converted to a number, and its high and low bytes are written to two consecutive memory cells starting at the given address.
display_state()	Reserved for printing the state of the machine.

4.8 InstructionRegister (Nabil)

The instruction register stores the current instruction as a four-character hexadecimal string and extracts its fields. A private helper, hexToInt(), converts one hexadecimal character to its number. Each accessor reads the digit that belongs to the field for the instruction format of the current op-code:

Member	Description
set(hex_str)	Stores the instruction string.
get_opcode()	The first digit.
get_r()	Register R: the second digit, except for MOVE (op-code 4), where it is the third digit.
get_s()	Register S: the third digit for op-codes 5 to 9, and the fourth digit for MOVE.
get_t()	Register T: the fourth digit for op-codes 5 to 9.
get_xy()	The 8-bit value or address formed by the third and fourth digits, for op-codes 1, 2, 3 and B.
get_x()	The rotation count: the fourth digit for ROTATE (op-code A).

4.9 CPU (Nabil)

The CPU groups the registers, the program counter, the instruction register, the ALU and the control unit.

Member	Description
step()	Runs one instruction through the control unit.
display_state()	Prints the program counter and the register table.

5. The Graphical User Interface

5.1 Overview

The file gui.cpp is a Qt Widgets application titled "Vole Machine Simulator (FCAI Cairo University)". A Machine structure owns every simulator component (program counter, instruction register, memory, registers, ALU, screen, control unit, CPU and voleMachine) and wires them together. The main window uses a dark theme, shows the state of the machine in separate panels, and has a toolbar and a menu bar with the same actions.

5.2 Layout

Figure 2 shows the layout of the main window.

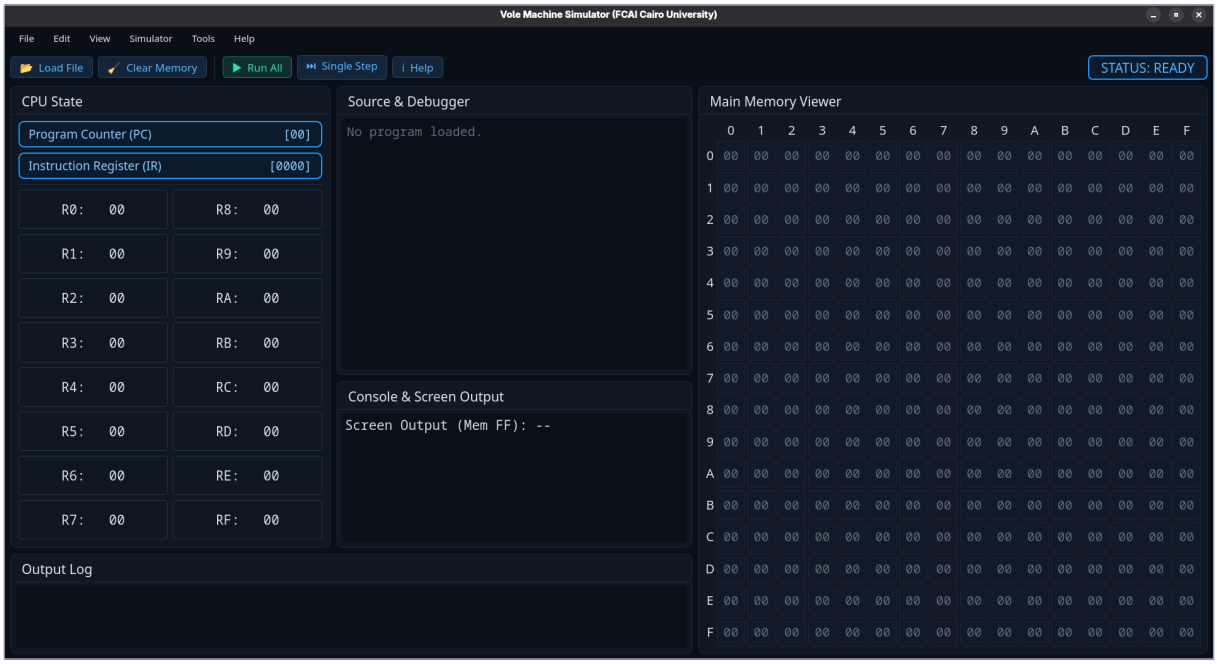


Figure 2. Layout of the main window.

5.3 Panels

Panel	Content
CPU State	The program counter, the instruction register, and all 16 registers.
Source & Debugger	The loaded program, one instruction per line with its address, the instruction in hexadecimal and a short comment describing it. An arrow marks the instruction that will execute next.
Console & Screen Output	The value most recently written to the screen.
Output Log	A step-by-step description of every executed instruction, and any errors.
Main Memory Viewer	A 16 x 16 grid of all 256 memory cells. Program bytes are highlighted, and the screen cell (FF) is shown separately.

5.4 Controls

Control	Action
Load File	Choose a program text file. The machine is reset, the program is loaded at address 00 and the number of instructions is written to the log.
Clear Memory	Reset the memory, registers, screen and program counter.
Run All	Run until the program halts. Execution stops after 100,000 steps to guard against infinite loops.
Single Step	Execute one instruction and update all panels.
Help	Shows the instruction set.
Status indicator	Shows READY, RUNNING, STEPPING or HALTED.

All actions are also available from the menu bar.

5.5 How the interface drives the simulator

- **Loading.** Loading a file resets the machine, calls voleMachine::load_program_from_file() at address 0, and finds the end of the program from the last non-zero memory byte.
- **Stepping.** Before each step the interface reads the next instruction bytes to show it in the instruction register and to write a description of it in the output log. It then calls CPU::step() and logs the new value of the program counter.
- **Screen output.** The interface captures the text printed by Screen::display() and shows the last value in the console panel and in the memory cell FF.
- **Source panel.** The comment for each instruction is generated from its op-code and operands, so the panel reads as a disassembly of the loaded program.
- **Safety.** If the program counter leaves memory, execution is stopped with an error in the log.

6. Program File Format

A program is a plain text file with one instruction per line, written as four hexadecimal digits. Blank lines and lines beginning with ";" are ignored. Text after the four digits on a line is ignored, so it can be used for comments. Instructions are loaded into memory from address 00.

7. Building and Running

Platform: Windows.

Requirements: Qt 5 or Qt 6 for Windows with the Widgets module, together with its MinGW (or MSVC) compiler. Both are installed by the Qt Online Installer, which also installs Qt Creator. All source files must be compiled together: Muhammad.cpp, Eslam.cpp, Nabil.cpp, Abdelrahman.cpp and gui.cpp.

7.1 Option 1 - Qt Creator (recommended)

- Open Qt Creator and choose File > New Project > Application (Qt) > Qt Widgets Application, or any empty Qt project.
- Add all the project's .cpp and .h files (Muhammad, Eslam, Nabil, Abdelrahman and gui.cpp) to the project.
- Make sure the project file contains `QT += widgets`.

- Select a Qt kit for Windows (MinGW or MSVC) and press Ctrl+R to build and run.

7.2 Option 2 - Command line (Qt for Windows, MinGW)

Create a file named VoleSimulator.pro in the project folder:

```
QT += widgets
CONFIG += c++17
SOURCES += Muhammad.cpp Eslam.cpp Nabil.cpp Abdelrahman.cpp gui.cpp
HEADERS += Muhammad.h Eslam.h Nabil.h Abdelrahman.h
```

Then open the Qt MinGW command prompt from the Start menu, go to the project folder and run:

```
qmake VoleSimulator.pro
mingw32-make
release\VoleSimulator.exe
```

If the program does not start because of missing DLLs, run `windeployqt release\VoleSimulator.exe` once to copy the Qt libraries next to it.

7.3 Using the simulator

1. Click **Load File** and choose a program text file.
2. Click **Single Step** to execute one instruction at a time, or **Run All** to run to completion.
3. Watch the registers, memory, screen output and log update.
4. Click **Clear Memory** to start again.

8. Testing

The following programs were run on the simulator core (the classes of Sections 4.1 to 4.9, driven one instruction at a time) to verify the instruction set and the execution engine. In every case the program ran until HALT and the values below are the ones observed.

Test 1: Worked example (LOAD, ADD, STORE to screen, HALT)

Adds 05 and 03 and shows the sum on the screen.

```
2105   ; Load immediate value 05 into register 1
2203   ; Load immediate value 03 into register 2
5312   ; ADD registers 1 and 2, put result in register 3
33FF   ; Store register 3 to screen (FF)
C000   ; Halt
```

Result: R1 = 05, R2 = 03, R3 = 08 after the third instruction; the screen shows 8; the machine halts after 5 steps.

Test 2: Check if an immediate value (07) is odd or even using bitwise AND with 01

The screen shows 1 if the value is odd and 0 if it is even.

```
; Muhammad Ahmed Noureldin Abdelghany ( Team Leader ) 20251324
; Nabil mohamed bahget mohamed 20250699
; Abdelrahman Haithem Mostafa Said 20250378
; Eslam Ahmed Abdelaziz Ahmed 20250083
; ------------------------------------------------------------
; Vole Machine Program
; Purpose: Check if 07 is odd or even using AND with 01.
; Screen shows 01 if odd, 00 if even.
; ------------------------------------------------------------

2107   ; Load immediate value 07 into register 1
2201   ; Load immediate value 01 into register 2
8312   ; AND registers 1 and 2, put result in register 3
33FF   ; Store register 3 to screen (FF)
C000   ; Halt
```

Result: R3 = 01 and the screen shows 1, so 07 is odd.

Test 3: Swap registers 1 and 2 (05 and 09) using XOR without a temporary register

Swaps the two values with three XOR instructions and displays both.

```
2105   ; Load immediate value 05 into register 1
2209   ; Load immediate value 09 into register 2
9112   ; R1 = R1 XOR R2  (05 XOR 09 = 0C)
9221   ; R2 = R2 XOR R1  (09 XOR 0C = 05)
9112   ; R1 = R1 XOR R2  (0C XOR 05 = 09)
31FF   ; Store register 1 to screen (FF)
32FF   ; Store register 2 to screen (FF)
C000   ; Halt
```

Result: The values of R1 after each XOR are 0C, 0C and 09, and R2 becomes 05. The screen shows 9 5, so the registers were swapped.

Test 4: ROTATE

Rotates 81 right by one step; the bit leaving the low end re-enters at the high end.

```
2181   ; Load immediate value 81 into register 1
A101   ; Rotate register 1 right by 1 step
31FF   ; Store register 1 to screen (FF)
C000   ; Halt
```

Result: R1 = C0 after the rotation and the screen shows 192 (C0 in hexadecimal).

Test 5: JUMP

Register 0 always equals itself, so B006 jumps to address 06 and skips the instruction at address 04.

```
2105   ; Load immediate value 05 into register 1
B006   ; Jump to address 06 because R0 = R0
31FF   ; (skipped) would show register 1
2209   ; Load immediate value 09 into register 2
32FF   ; Store register 2 to screen (FF)
C000   ; Halt
```

Result: After the jump the program counter is 06 and the skipped store never runs. The screen shows only 9 and the machine halts after 5 steps.

Test 6: Floating-point ADD

Adds 7A (5.0) and 3C (0.375) in floating-point format.

```
207A   ; Load immediate value 7A (5.0) into register 0
213C   ; Load immediate value 3C (0.375) into register 1
6201   ; Floating-point ADD of R0 and R1 into R2
32FF   ; Store register 2 to screen (FF)
C000   ; Halt
```

Result: The exact sum is 5.375, which needs more than four mantissa bits, so it is truncated to 7A (5.0). R2 = 7A and the screen shows 122 (7A in decimal). This shows the limited precision of the 8-bit floating-point format.

9. Work Breakdown Structure

The table below outlines the responsibilities and contributions of each team member toward the completion of the Vole Machine Simulator project.

Team Member	Role / Responsibilities & Contributions
Muhammad Ahmed Noureldin Abdelghany (Team Leader)	• Created the GitHub repository • Created the project plan • Implemented the ALU class • Implemented the Control Unit (CU) class • Implemented the graphical user interface (gui.cpp)
Eslam Ahmed Abdelaziz Ahmed	• Implemented the Memory class • Implemented the Screen class
Abdelrahman Haithem Mostafa Said	• Implemented the programCounter class • Implemented the registers class • Implemented the voleMachine class (loading programs from a file)
Nabil Mohamed Bahget Mohamed	• Implemented the InstructionRegister class • Implemented the CPU class • Created both project videos (two videos in total)
Whole team	• Team meetings and the final code review before submission • README files (.txt and .md)

Development plan

The plan below was created by Muhammad and followed by the team.

Date	Task
Fri 30/10/2026	ALU, Memory, Program Counter, Instruction Register
Sat 31/10/2026	Control Unit, Screen, Registers
Mon 2/11/2026	CPU
Tue 3/11/2026	GUI, VoleMachine
Thu 5/11/2026	README (.txt) and (.md), video
Fri 6/11/2026	Final code review and submission

10. Conclusion

The team implemented a complete Vole Machine Simulator in C++ with a Qt graphical interface. Each hardware part of the machine is a separate class: the ALU and control unit execute the twelve instructions, the memory, registers, program counter and instruction register hold the state of the machine, the screen collects the output, the CPU groups the processing parts, and voleMachine loads programs from text files. The graphical interface shows the registers, memory, source and screen and lets the user load a program, step through it or run it to the end. The test programs for loading and storing, arithmetic, logic, rotation, jumping and floating-point addition all ran to HALT and produced the expected screen output.